Dr. RVR
INSTITUTE OF TECHNOLOGY
DEEMED TO BE UNIVERSITY

DR.RVR NRI INSTITUTE OF TECHNOLOGY DEEMED TO BE UNIVERSITY

Department of Computer Science and Engineering

ACTIVITY-BASED LEARNING REPORT

PYTHON PROGRAMMING



Submitted in partial fulfillment of the Web Development Activity-Based Learning assignment

Student Name:	**Suma Sri Pasupuleti**
Roll Number:	**26000050**
Branch / Section:	**CSE 1**
Academic Year:	**2026–2027**
Faculty Name:	**Venu Gopal Sir**

Case Study 1: ATM Transaction Sequence Simulation

1. Problem Statement

An ATM transaction system allows a user to perform a sequence of deposits and withdrawals. The program should maintain the account balance, validate each transaction, enforce withdrawal limits, reject invalid transactions, and display a summary of all transactions at the end.

The program will simulate ATM operations without connecting to a real bank account.

2. Objectives

- To understand how ATM transactions work.
- To accept a sequence of deposits and withdrawals.
- To update the account balance after each valid transaction.
- To reject invalid amounts and withdrawals exceeding the available balance.
- To apply a withdrawal limit.
- To display a final transaction summary.

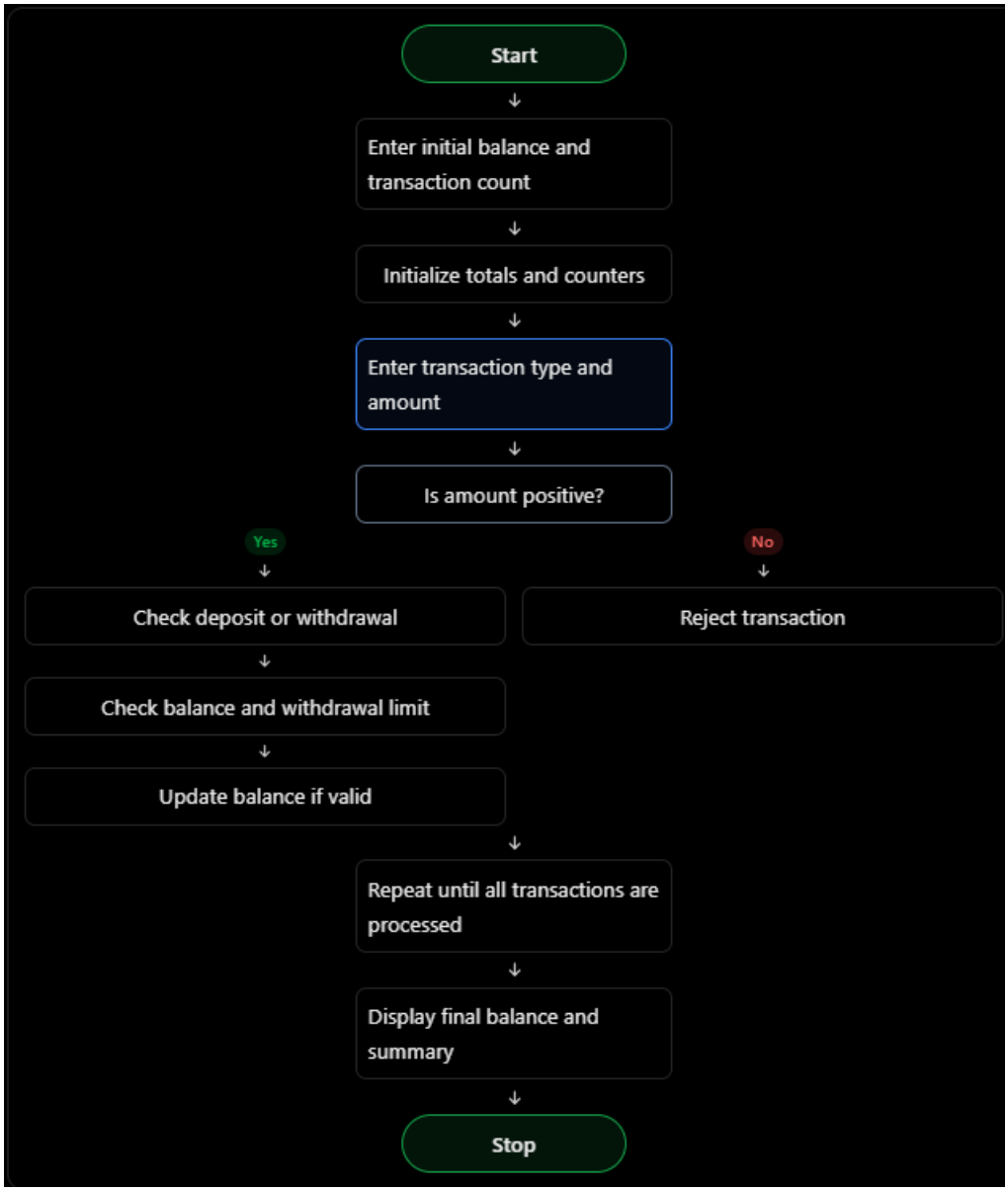
3. Assumptions

1. The initial account balance is entered by the user.
2. The user specifies how many transactions they want to perform.
3. Each transaction is either a deposit (D) or withdrawal (W).
4. Transaction amounts must be greater than zero.
5. A withdrawal cannot exceed the available balance.
6. The maximum amount allowed for one withdrawal is ₹10,000.
7. This is a simulation for educational purposes only.

4. Algorithm

1. Start the program.
2. Enter the initial account balance.
3. Enter the number of transactions to perform.
4. Initialize the deposit total, withdrawal total, and valid/invalid transaction counters to zero.
5. Repeat the following steps for each transaction:
    - Enter the transaction type (D for deposit or W for withdrawal).
    - Enter the transaction amount.
    - Check whether the amount is positive.
    - If the transaction is a deposit, add the amount to the balance.
    - If it is a withdrawal, check the withdrawal limit and available balance.
    - If valid, update the balance and transaction totals.
    - Otherwise, reject the transaction and display a message.
6. Display the final account balance and transaction summary.
7. Stop the program.

5. Flowchart


Start
Enter initial balance and transaction count
Initialize totals and counters
Enter transaction type and amount
Is amount positive?
Yes
No
Check deposit or withdrawal
Reject transaction
Check balance and withdrawal limit
Update balance if valid
Repeat until all transactions are processed
Display final balance and summary
Stop

6. Pseudocode

```
START

INPUT initial_balance
INPUT number_of_transactions

SET total_deposits = 0
SET total_withdrawals = 0
SET valid_transactions = 0
SET invalid_transactions = 0

REPEAT for each transaction:

    INPUT transaction_type
    INPUT amount

    IF amount <= 0:
        DISPLAY "Invalid amount"
        INCREASE invalid_transactions

    ELSE IF transaction_type is "D":
        ADD amount to balance
        ADD amount to total_deposits
        INCREASE valid_transactions

    ELSE IF transaction_type is "W":
        IF amount > 10000:
            DISPLAY "Withdrawal limit exceeded"
            INCREASE invalid_transactions
```

```
        ELSE IF amount > balance:
            DISPLAY "Insufficient balance"
            INCREASE invalid_transactions

        ELSE:
            SUBTRACT amount from balance
            ADD amount to total_withdrawals
            INCREASE valid_transactions

    ELSE:
        DISPLAY "Invalid transaction type"
        INCREASE invalid_transactions

DISPLAY final balance
DISPLAY total deposits
DISPLAY total withdrawals
DISPLAY valid transaction count
DISPLAY invalid transaction count

STOP
```

7.Python Source Code

```python
balance = float(input("Enter initial account balance (Rs.): "))
number_of_transactions = int(input("Enter number of transactions: "))

total_deposits = 0
total_withdrawals = 0
valid_transactions = 0
invalid_transactions = 0

if balance < 0 or number_of_transactions < 0:
    print("Invalid initial balance or transaction count.")

else:
    for i in range(number_of_transactions):
        print("\nTransaction", i + 1)

        transaction_type = input("Enter D for Deposit or W for Withdrawal: ")
        transaction_type = transaction_type.upper()

        amount = float(input("Enter transaction amount (Rs.): "))

        if amount <= 0:
            print("Invalid amount. Amount must be greater than zero.")
            invalid_transactions = invalid_transactions + 1

        elif transaction_type == "D":
            balance = balance + amount
            total_deposits = total_deposits + amount
            valid_transactions = valid_transactions + 1
            print("Deposit successful.")

        elif transaction_type == "W":
            if amount > 10000:
                print("Withdrawal limit exceeded.")
                invalid_transactions = invalid_transactions + 1

            elif amount > balance:
                print("Insufficient balance.")
                invalid_transactions = invalid_transactions + 1

            else:
                balance = balance - amount
                total_withdrawals = total_withdrawals + amount
                valid_transactions = valid_transactions + 1
                print("Withdrawal successful.")

        else:
            print("Invalid transaction type.")
            invalid_transactions = invalid_transactions + 1

    print("\n--- ATM TRANSACTION SUMMARY ---")
    print("Final account balance: Rs.", round(balance, 2))
    print("Total deposits: Rs.", round(total_deposits, 2))
    print("Total withdrawals: Rs.", round(total_withdrawals, 2))
    print("Valid transactions:", valid_transactions)
    print("Invalid transactions:", invalid_transactions)
```

8.Test Cases

Test	Initial balance	Transactions	Expected result
1	₹5,000	Deposit ₹2,000	Balance ₹7,000
2	₹5,000	Withdraw ₹1,500	Balance ₹3,500
3	₹5,000	Withdraw ₹6,000	Rejected: insufficient balance
4	₹20,000	Withdraw ₹12,000	Rejected: withdrawal limit exceeded
5	₹5,000	Deposit ₹0	Rejected: invalid amount
6	₹5,000	Enter transaction type X	Rejected: invalid transaction type

9.Output Screenshots

Test Case 1

```
Enter initial account balance (Rs.): 5000
Enter number of transactions: 1

Transaction 1
Enter D for Deposit or W for Withdrawal: D
Enter transaction amount (Rs.): 2000
Deposit successful.

--- ATM TRANSACTION SUMMARY ---
Final account balance: Rs. 7000.0
Total deposits: Rs. 2000.0
Total withdrawals: Rs. 0
Valid transactions: 1
Invalid transactions: 0
```

Test Case 2

```
Enter initial account balance (Rs.): 5000
Enter number of transactions: 1

Transaction 1
Enter D for Deposit or W for Withdrawal: W
Enter transaction amount (Rs.): 1500
Withdrawal successful.

--- ATM TRANSACTION SUMMARY ---
Final account balance: Rs. 3500.0
Total deposits: Rs. 0
Total withdrawals: Rs. 1500.0
Valid transactions: 1
Invalid transactions: 0
```

Test Case 3

```
Enter initial account balance (Rs.): 5000
Enter number of transactions: 1

Transaction 1
Enter D for Deposit or W for Withdrawal: W
Enter transaction amount (Rs.): 6000
Insufficient balance.

--- ATM TRANSACTION SUMMARY ---
Final account balance: Rs. 5000.0
Total deposits: Rs. 0
Total withdrawals: Rs. 0
Valid transactions: 0
Invalid transactions: 1
```

10. Conclusion

The ATM Transaction Sequence Simulation demonstrates how Python can be used to simulate basic banking transactions. The program accepts deposits and withdrawals, updates the account balance, applies a withdrawal limit, rejects invalid transactions, and displays a transaction summary. It uses variables, arithmetic operators, conditional statements, loops, and string operations to implement the required logic.

This case study helped demonstrate the use of basic Python programming concepts to solve a practical problem.

Case Study 2: Multi-Product Shopping Budget Constraint

1. Problem Statement

A student has a fixed shopping budget and wants to purchase multiple products. The program accepts the price and quantity of each product, calculates the total cost, and compares it with the available budget.

If the total cost exceeds the budget, the program identifies the amount by which the budget is exceeded and helps the student decide which purchases could be reduced or removed to stay within the budget.

2. Objectives

- To accept the shopping budget from the user.
- To enter product names, prices, and quantities.
- To calculate the cost of each product.
- To calculate the total shopping cost.
- To compare the total cost with the available budget.
- To identify whether the shopping plan is within budget or exceeds it.
- To help the user adjust purchases when necessary.

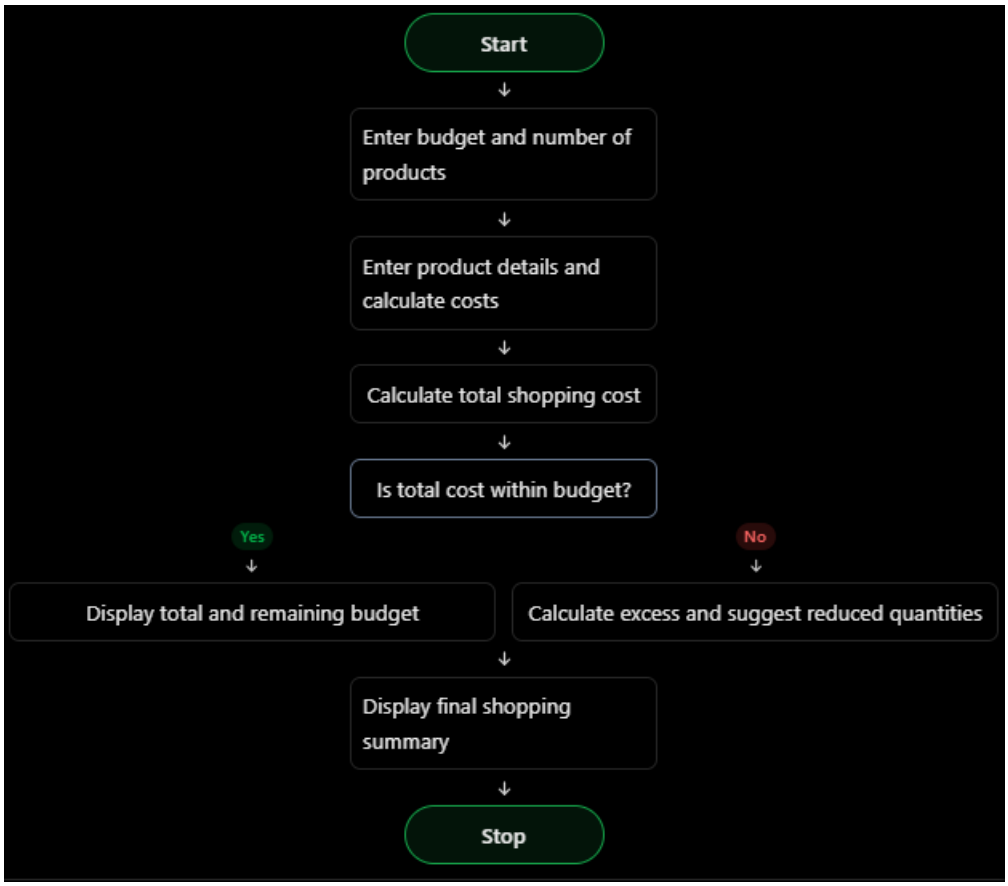
3. Assumptions

1. The user enters a fixed shopping budget.
2. The number of products is entered before the shopping list.
3. Each product has a name, price, and quantity.
4. Product prices must be greater than zero.
5. Quantities must be greater than zero.
6. The program uses a simple method to reduce purchases if the total exceeds the budget.
7. This program is a shopping-cost simulation and does not make actual purchases.

4. Algorithm

1. Start the program.
2. Enter the shopping budget.
3. Enter the number of products.
4. Create lists to store product names, prices, and quantities.
5. For each product:
    - Enter the product name.
    - Enter its price and quantity.
    - Calculate the product cost.
    - Add the cost to the total shopping cost.
6. Display the cost of each product and the total cost.
7. Compare the total cost with the budget.
8. If the total is within budget, display the remaining balance.
9. If the total exceeds the budget:
    - Calculate the amount exceeding the budget.
    - Suggest a revised shopping plan by considering products in the order entered.
    - Calculate how many units of each product can fit within the remaining budget.
10. Display the revised shopping plan and its total cost.
11. Stop the program.

5.Flow Chart



6.Pseudo Code

START

INPUT budget

INPUT number_of_products

SET total_cost = 0

CREATE lists for names, prices, and quantities

REPEAT for each product:

    INPUT product_name

    INPUT product_price

    INPUT product_quantity

```
    CALCULATE product_cost
    ADD product_cost to total_cost

    STORE product details in lists

DISPLAY total_cost

IF total_cost <= budget:
    DISPLAY "Shopping is within budget"
    DISPLAY remaining budget

ELSE:
    CALCULATE excess_amount
    DISPLAY "Budget exceeded"
    SET remaining_budget = budget

    FOR each product:
        CALCULATE affordable quantity
        SELECT quantity that fits remaining budget
        UPDATE remaining budget

    DISPLAY revised shopping quantities
    DISPLAY revised total cost

STOP
```

7.Python Source Code

```
budget = float(input("Enter your shopping budget (Rs.): "))
number_of_products = int(input("Enter number of products: "))

names = []
prices = []
quantities = []
costs = []

total_cost = 0

if budget < 0 or number_of_products < 0:
    print("Invalid budget or number of products.")

else:
    for i in range(number_of_products):
        print("\nProduct", i + 1)

        name = input("Enter product name: ")
        price = float(input("Enter price per unit (Rs.): "))
        quantity = int(input("Enter quantity: "))

        if price <= 0 or quantity <= 0:
            print("Invalid price or quantity.")
            print("Please run the program again with valid values.")
            break

        product_cost = price * quantity

        names.append(name)
        prices.append(price)
        quantities.append(quantity)
        costs.append(product_cost)

        total_cost = total_cost + product_cost

    if len(names) == number_of_products:
        print("\n--- SHOPPING BILL ---")

        for i in range(number_of_products):
            print(names[i], "- Rs.", round(costs[i], 2))

        print("Total shopping cost: Rs.", round(total_cost, 2))
        print("Available budget: Rs.", round(budget, 2))

        if total_cost <= budget:
            print("Shopping is within budget.")
            print("Remaining budget: Rs.", round(budget - total_cost, 2))

        else:
            excess_amount = total_cost - budget

            print("Budget exceeded by: Rs.", round(excess_amount, 2))
            print("\n--- SUGGESTED SHOPPING PLAN ---")
            print("Products are considered in the order entered.")

            remaining_budget = budget
            revised_total = 0


     for i in range(number_of_products):
         affordable_quantity = int(remaining_budget // prices[i])

         if affordable_quantity > quantities[i]:
             affordable_quantity = quantities[i]

         revised_cost = prices[i] * affordable_quantity

         print(names[i], "- Original quantity:", quantities[i])
         print("Suggested quantity:", affordable_quantity)
         print("Suggested cost: Rs.", round(revised_cost, 2))

         remaining_budget = remaining_budget - revised_cost
         revised_total = revised_total + revised_cost

     print("Revised shopping total: Rs.", round(revised_total, 2))
     print("Budget remaining: Rs.", round(remaining_budget, 2))
```

8. Test Cases

Test	Budget	Shopping details	Expected result
1	₹1,000	Notebook ₹100 × 3; Pen ₹20 × 5	Total ₹400; within budget
2	₹1,000	Notebook ₹100 × 3; Bag ₹500 × 2; Pen ₹20 × 5	Total ₹1,400; budget exceeded by ₹400
3	₹500	Book ₹200 × 2; Pen ₹10 × 5	Total ₹450; within budget
4	₹100	Bag ₹250 × 1	Budget exceeded; suggested quantity is 0
5	₹500	Product quantity entered as 0	Invalid quantity message

9.Output Screenshots

Test Case 1

```
Enter your shopping budget (Rs.): 1000
Enter number of products: 2

Product 1
Enter product name: Note books
Enter price per unit (Rs.): 100
Enter quantity: 3

Product 2
Enter product name: Pens
Enter price per unit (Rs.): 20
Enter quantity: 5

--- SHOPPING BILL ---
Note books - Rs. 300.0
Pens - Rs. 100.0
Total shopping cost: Rs. 400.0
Available budget: Rs. 1000.0
Shopping is within budget.
Remaining budget: Rs. 600.0
```

Test Case 2

```
Enter your shopping budget (Rs.): 1000
Enter number of products: 3

Product 1
Enter product name: Note books
Enter price per unit (Rs.): 100
Enter quantity: 3

Product 2
Enter product name: Bag
Enter price per unit (Rs.): 500
Enter quantity: 2

Product 3
Enter product name: Pen
Enter price per unit (Rs.): 20
Enter quantity: 5

--- SHOPPING BILL ---
Note books  - Rs. 300.0
Bag - Rs. 1000.0
Pen - Rs. 100.0
Total shopping cost: Rs. 1400.0
Available budget: Rs. 1000.0
Budget exceeded by: Rs. 400.0

--- SUGGESTED SHOPPING PLAN ---
Products are considered in the order entered.
Note books  - Original quantity: 3
Suggested quantity: 3
Suggested cost: Rs. 300.0
Bag - Original quantity: 2
Suggested quantity: 1
Suggested cost: Rs. 500.0
Pen - Original quantity: 5
Suggested quantity: 5
Suggested cost: Rs. 100.0
Revised shopping total: Rs. 900.0
Budget remaining: Rs. 100.0
```

10. Conclusion

The Multi-Product Shopping Budget Constraint program demonstrates how Python can help manage shopping expenses within a fixed budget. It accepts product details, calculates individual and total costs, identifies budget overruns, and suggests revised quantities to reduce spending.

This case study uses variables, arithmetic operators, conditional statements, loops, and lists to solve a practical budgeting problem.

Case Study 3: Password Policy Validation Engine

1. Problem Statement

Passwords are used to protect user accounts and personal information. A password policy defines rules that a password must satisfy, such as minimum length, character categories, repeated characters, and prohibited substrings.

The objective is to develop a Python program that checks a sample password against a set of specified rules and explains each condition that the password fails.

This program performs a basic local validation. It does not connect to any website or account.

2. Objectives

- To accept a sample password from the user.
- To check whether the password meets the minimum length requirement.
- To check for uppercase letters, lowercase letters, digits, and special characters.
- To detect repeated consecutive characters.
- To check for a prohibited substring.
- To display every rule that the password fails.
- To determine whether the password satisfies all the defined rules.

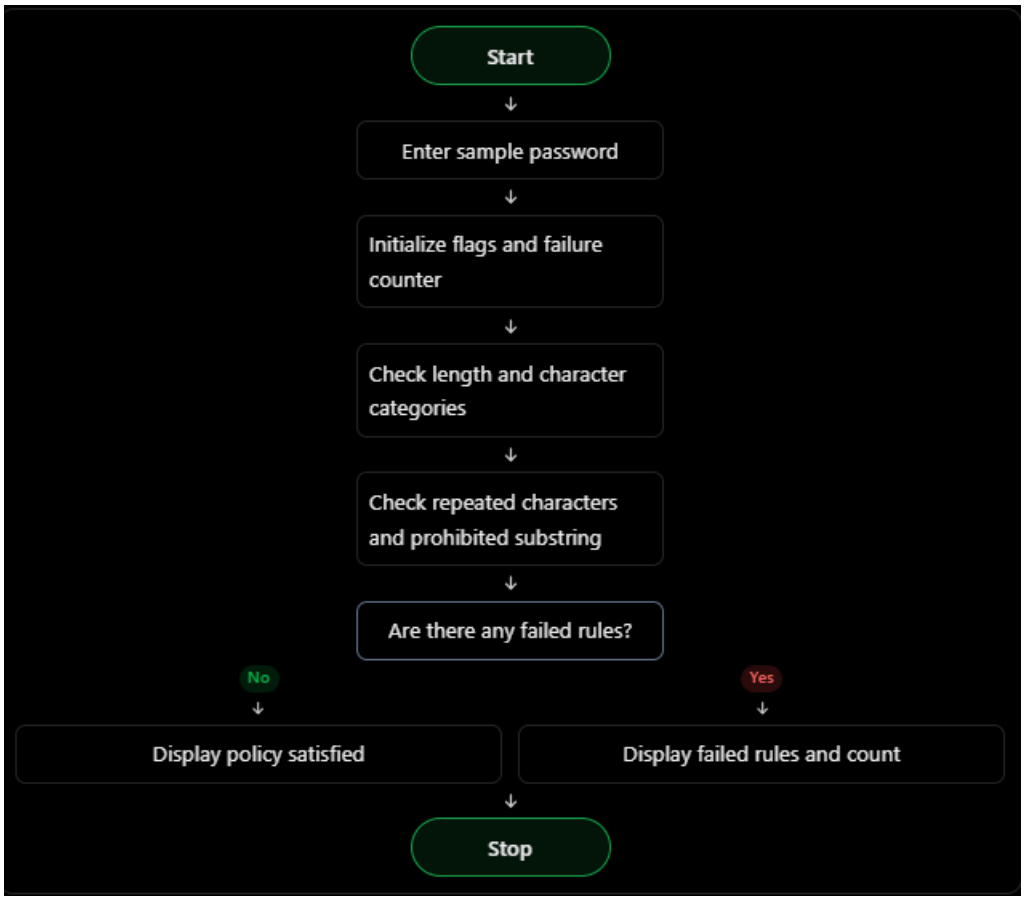
3. Assumptions

1. The minimum password length is 8 characters.
2. A valid password must contain at least one uppercase letter.
3. A valid password must contain at least one lowercase letter.
4. A valid password must contain at least one digit.
5. A valid password must contain at least one special character.
6. Three consecutive identical characters are not allowed.
7. The substring password is prohibited, regardless of letter case.
8. The program checks only the rules defined in this assignment. Passing these checks does not guarantee that a password is secure.

4. Algorithm

1. Start the program.
2. Accept a sample password from the user.
3. Initialize a counter for failed rules.
4. Check whether the password has at least 8 characters.
5. Check whether it contains uppercase letters.
6. Check whether it contains lowercase letters.
7. Check whether it contains digits.
8. Check whether it contains at least one special character.
9. Check whether three consecutive characters are identical.
10. Check whether the password contains the prohibited substring password, ignoring case.
11. Display a message for every failed rule.
12. If no rules fail, display that the password satisfies the defined policy.
13. Otherwise, display the total number of failed rules.
14. Stop the program.

5.Flow Chart



6. Pseudocode

START

INPUT password

SET failed_rules = 0

IF password length is less than 8:

    DISPLAY minimum length failure

    INCREASE failed_rules

SET uppercase_found = false

SET lowercase_found = false

SET digit_found = false

```
SET special_found = false

FOR each character in password:
    CHECK uppercase letter
    CHECK lowercase letter
    CHECK digit
    CHECK special character

IF uppercase_found is false:
    DISPLAY uppercase failure
    INCREASE failed_rules

IF lowercase_found is false:
    DISPLAY lowercase failure
    INCREASE failed_rules

IF digit_found is false:
    DISPLAY digit failure
    INCREASE failed_rules

IF special_found is false:
    DISPLAY special character failure
    INCREASE failed_rules

CHECK for three consecutive identical characters

IF found:
    DISPLAY repeated character failure
    INCREASE failed_rules
```

```
CONVERT password to lowercase

IF password contains "password":
    DISPLAY prohibited substring failure
    INCREASE failed_rules

IF failed_rules equals 0:
    DISPLAY "Password satisfies the defined policy"
ELSE:
    DISPLAY number of failed rules

STOP
```

7.Python Source Code

```
password = input("Enter a sample password to check: ")

failed_rules = 0

uppercase_found = False
lowercase_found = False
digit_found = False
special_found = False

print("\n--- PASSWORD POLICY VALIDATION ---")

if len(password) < 8:
    print("Failed: Password must contain at least 8 characters.")
    failed_rules = failed_rules + 1

for character in password:
    if character.isupper():
        uppercase_found = True

    if character.islower():
        lowercase_found = True

    if character.isdigit():
        digit_found = True

    if character in "!@#$%^&*()-_+=?":
        special_found = True

if uppercase_found == False:
    print("Failed: At least one uppercase letter is required.")
    failed_rules = failed_rules + 1

if lowercase_found == False:
    print("Failed: At least one lowercase letter is required.")
    failed_rules = failed_rules + 1

if digit_found == False:
    print("Failed: At least one digit is required.")
    failed_rules = failed_rules + 1

if special_found == False:
    print("Failed: At least one special character is required.")
    failed_rules = failed_rules + 1

repeated_found = False

for i in range(len(password) - 2):
    if password[i] == password[i + 1] and password[i] == password[i + 2]:
        repeated_found = True

if repeated_found == True:
    print("Failed: Three consecutive identical characters are not allowed.")
    failed_rules = failed_rules + 1

if "password" in password.lower():
    print("Failed: The word 'password' is not allowed.")
    failed_rules = failed_rules + 1


if failed_rules == 0:
    print("Password satisfies all the defined policy rules.")
else:
    print("Total failed rules:", failed_rules)

print("Validation completed.")
```

8. Test Cases

Test	Sample password	Expected result
1	Abc123!x	Passes all defined rules
2	abc123	Fails minimum length, uppercase, and special character rules
3	ABCDEFG1!	Fails lowercase requirement
4	abcdefg!	Fails uppercase and digit requirements
5	Abc111!x	Fails repeated-character rule
6	PassWORD123!	Fails prohibited substring rule

9.Output screenshots of all the test cases

```
Enter a sample password to check: Abc123!x

--- PASSWORD POLICY VALIDATION ---
Password satisfies all the defined policy rules.
Validation completed.

========== RESTART:
Enter a sample password to check: abc123

--- PASSWORD POLICY VALIDATION ---
Failed: Password must contain at least 8 characters.
Failed: At least one uppercase letter is required.
Failed: At least one special character is required.
Total failed rules: 3
Validation completed.

========== RESTART:
Enter a sample password to check: ABCDEFG1!

--- PASSWORD POLICY VALIDATION ---
Failed: At least one lowercase letter is required.
Total failed rules: 1
Validation completed.

========== RESTART:
Enter a sample password to check: abcdefg!

--- PASSWORD POLICY VALIDATION ---
Failed: At least one uppercase letter is required.
Failed: At least one digit is required.
Total failed rules: 2
Validation completed.

========== RESTART:
Enter a sample password to check: Abc111!x

--- PASSWORD POLICY VALIDATION ---
Failed: Three consecutive identical characters are not allowed.
Total failed rules: 1
Validation completed.

========== RESTART:
Enter a sample password to check: PassWORD123!

--- PASSWORD POLICY VALIDATION ---
Failed: The word 'password' is not allowed.
Total failed rules: 1
Validation completed.
```

10. Conclusion

The Password Policy Validation Engine demonstrates how Python can validate a password against predefined rules. It checks password length, character categories, repeated consecutive characters, and prohibited substrings. It also explains each failed condition instead of simply displaying a pass or fail result.

This case study demonstrates the use of strings, loops, variables, conditional statements, and basic string operations to solve a practical validation problem.

Case Study 4: Attendance Recovery Computation

1. Problem Statement

Students are often required to maintain a minimum attendance percentage to be eligible for examinations. This program calculates a student's current attendance percentage and determines how many consecutive classes the student must attend without missing any further classes to reach the required attendance threshold.

The program also checks different future attendance scenarios and displays whether the student currently meets the attendance requirement.

2. Objectives

- To accept the number of classes conducted and classes attended.
- To calculate the current attendance percentage.
- To compare current attendance with the required threshold.
- To calculate the minimum additional consecutive classes required to reach the threshold.
- To validate the input values.
- To display the attendance status clearly.

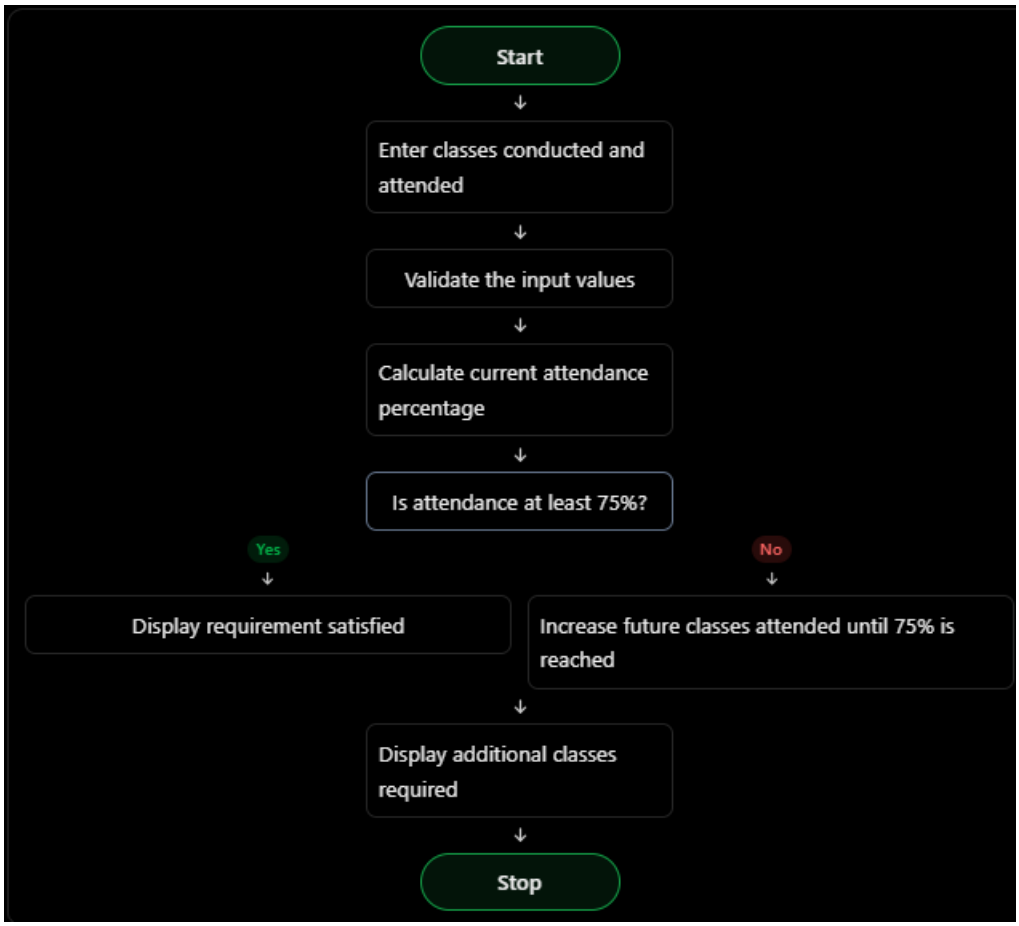
3. Assumptions

1. The number of classes conducted must be greater than zero.
2. The number of classes attended cannot be negative or greater than the number conducted.
3. The required attendance percentage is 75%.
4. Future classes are attended without any absences.
5. The program calculates the minimum number of additional classes required to reach the threshold.
6. This program is an educational simulation; official attendance rules may differ.

4. Algorithm

1. Start the program.
2. Enter the total number of classes conducted.
3. Enter the number of classes attended.
4. Validate the input values.
5. Calculate the current attendance percentage.
6. Compare the percentage with the required attendance threshold of 75%.
7. If the student already meets the threshold, display that the attendance requirement is satisfied.
8. Otherwise, initialize the additional class count to zero.
9. Use a loop to increase the number of future classes attended until the attendance percentage reaches 75%.
10. Display the minimum number of consecutive classes that must be attended.
11. Stop the program.

5.Flow chart



6. Pseudocode

START

INPUT total_classes

INPUT attended_classes

IF total_classes <= 0:

    DISPLAY "Invalid total classes"

ELSE IF attended_classes < 0 OR attended_classes > total_classes:

    DISPLAY "Invalid attended classes"

```
ELSE:
    CALCULATE attendance_percentage

    DISPLAY current attendance percentage

    IF attendance_percentage >= 75:
        DISPLAY "Attendance requirement satisfied"

    ELSE:
        SET future_classes = 0
        SET future_total = total_classes
        SET future_attended = attended_classes

        WHILE attendance percentage is less than 75:
            INCREASE future_classes
            INCREASE future_total
            INCREASE future_attended

        DISPLAY future_classes
        DISPLAY "Attend these classes consecutively"

STOP
```

7. Python Source Code

```
total_classes = int(input("Enter total number of classes conducted: "))
attended_classes = int(input("Enter number of classes attended: "))

required_percentage = 75

if total_classes <= 0:
    print("Invalid input. Total classes must be greater than zero.")

elif attended_classes < 0 or attended_classes > total_classes:
    print("Invalid input. Check the number of classes attended.")

else:
    attendance_percentage = (attended_classes / total_classes) * 100

    print("\n--- ATTENDANCE REPORT ---")
    print("Total classes conducted:", total_classes)
    print("Classes attended:", attended_classes)
    print("Current attendance:", round(attendance_percentage, 2), "%")

    if attendance_percentage >= required_percentage:
        print("Attendance requirement satisfied.")
        print("Additional classes required: 0")

    else:
        future_classes = 0
        future_total = total_classes
        future_attended = attended_classes

        while (future_attended / future_total) * 100 < required_percentage:
            future_classes = future_classes + 1
            future_total = future_total + 1
            future_attended = future_attended + 1

        print("Attendance requirement not yet satisfied.")
        print("Minimum consecutive classes to attend:", future_classes)

        final_percentage = (future_attended / future_total) * 100
        print("Attendance after attending these classes:",
              round(final_percentage, 2), "%")
```

The key formula is:

$$\text{Attendance Percentage} = \frac{\text{Classes Attended}}{\text{Total Classes Conducted}} \times 100$$

8. Test Cases

Test	Total classes	Attended classes	Expected result
1	100	90	Attendance satisfied; 0 additional classes
2	100	60	Attend 60 consecutive classes to reach 75%
3	80	56	Attend 16 consecutive classes to reach 75%
4	40	30	Attendance satisfied; 0 additional classes
5	0	0	Invalid total classes
6	50	55	Invalid attended classes

9.Output screenshot of all testcases

```
== RESTART: C:/Users/Sumasri/AppData/Local/Programs/Python/Python313/case
Enter total number of classes conducted: 100
Enter number of classes attended: 90

--- ATTENDANCE REPORT ---
Total classes conducted: 100
Classes attended: 90
Current attendance: 90.0 %
Attendance requirement satisfied.
Additional classes required: 0

================================================================================
Enter total number of classes conducted: 100
Enter number of classes attended: 60

--- ATTENDANCE REPORT ---
Total classes conducted: 100
Classes attended: 60
Current attendance: 60.0 %
Attendance requirement not yet satisfied.
Minimum consecutive classes to attend: 60
Attendance after attending these classes: 75.0 %

================================================================================
Enter total number of classes conducted: 80
Enter number of classes attended: 56

--- ATTENDANCE REPORT ---
Total classes conducted: 80
Classes attended: 56
Current attendance: 70.0 %
Attendance requirement not yet satisfied.
Minimum consecutive classes to attend: 16
Attendance after attending these classes: 75.0 %

================================================================================
Enter total number of classes conducted: 40
Enter number of classes attended: 30

--- ATTENDANCE REPORT ---
Total classes conducted: 40
Classes attended: 30
Current attendance: 75.0 %
Attendance requirement satisfied.
Additional classes required: 0

================================================================================
Enter total number of classes conducted: 0
Enter number of classes attended: 0
Invalid input. Total classes must be greater than zero.

================================================================================
Enter total number of classes conducted: 50
Enter number of classes attended: 55
Invalid input. Check the number of classes attended.
```

10. Conclusion

The Attendance Recovery Computation program calculates a student's current attendance percentage and determines the minimum number of consecutive classes required to reach a specified attendance threshold. It validates input values and uses conditional statements and a loop to calculate the result.

This case study demonstrates the practical use of variables, arithmetic operators, conditional statements, and loops in Python.

Case Study 5: Student Performance Trend Analysis

1. Problem Statement

Student performance may change across multiple examinations. Teachers need a simple system to record marks, compare performance across examinations, identify improving or declining performance, and find subjects in which a student consistently scores low marks.

The aim of this program is to develop a Python-based Student Performance Trend Analysis system that records marks in three examinations, calculates average marks, identifies performance trends, and highlights subjects where marks remain below the pass threshold.

2. Objectives

- To accept subject names and marks for three examinations.
- To calculate the average marks for each subject.
- To compare marks from the first examination to the third examination.
- To identify improving, declining, or unchanged performance.
- To identify consistently weak subjects.
- To display a final performance summary.

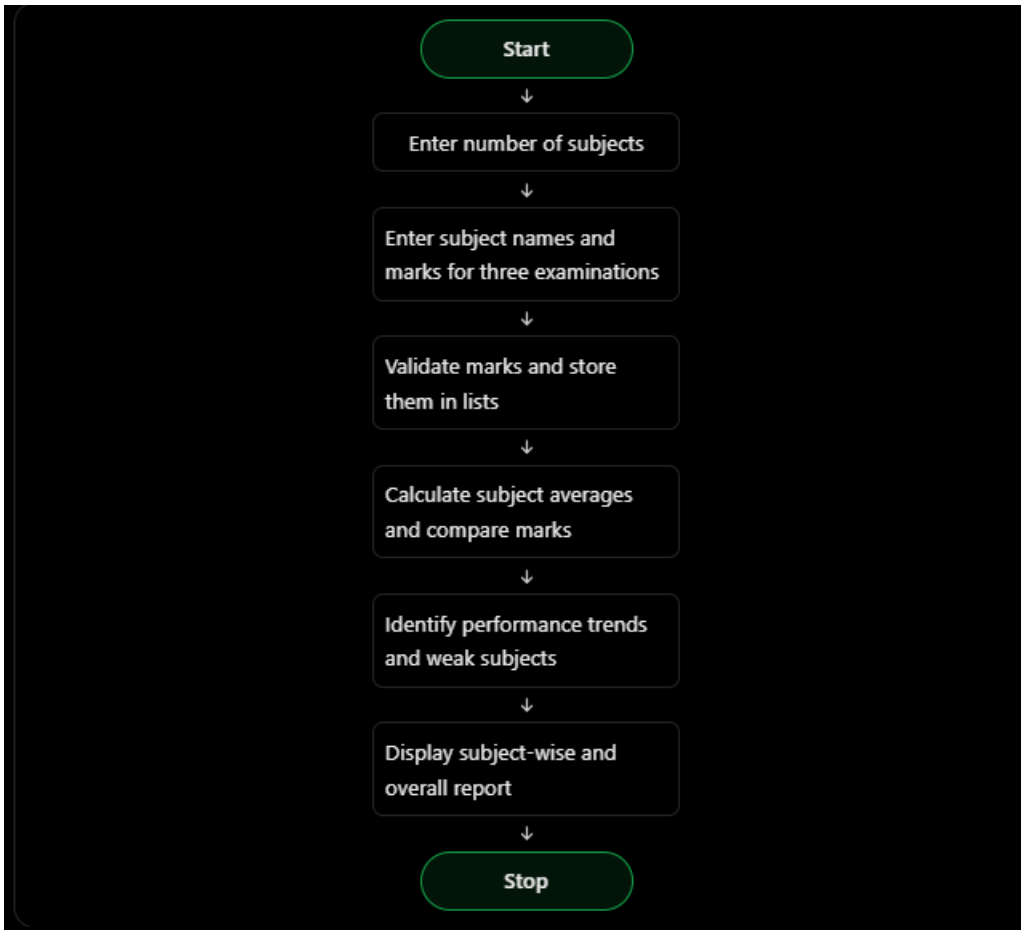
3. Assumptions

1. The student is evaluated in multiple subjects.
2. Marks are entered for three examinations.
3. Each examination mark must be between 0 and 100.
4. A mark below 40 is considered low for this program.
5. A subject is classified as consistently weak if all three examination marks are below 40.
6. Performance is improving if the third examination mark is greater than the first; declining if it is lower; otherwise, it is unchanged.
7. The program uses basic Python concepts such as variables, lists, loops, conditional statements, arithmetic operations, and input/output.

4. Algorithm

1. Start the program.
2. Read the number of subjects.
3. Check that the number of subjects is greater than zero.
4. Create lists to store subject names and marks from three examinations.
5. For each subject:
    - Read the subject name.
    - Read marks from Examination 1, Examination 2, and Examination 3.
    - Check whether each mark is between 0 and 100.
    - Store the valid information in lists.
6. Calculate the average mark for each subject.
7. Compare the first and third examination marks.
8. Identify whether performance is improving, declining, or unchanged.
9. Check whether all three marks are below 40.
10. Display the performance report for every subject.
11. Calculate and display the overall average for each examination.
12. Stop the program.

5.Flow chart



6. Pseudocode

START

INPUT number_of_subjects

IF number_of_subjects <= 0

    DISPLAY "Invalid number of subjects"

ELSE

    CREATE lists for subject names and three exam marks

    SET totals for all examinations to zero

    FOR each subject

        INPUT subject name

```
    INPUT marks for Exam 1, Exam 2, Exam 3

    IF any mark is below 0 or above 100
        DISPLAY "Invalid marks"
        STOP
    ELSE
        STORE subject name and marks
        ADD marks to examination totals
    END IF
END FOR

FOR each subject
    CALCULATE average marks

    IF Exam 3 marks > Exam 1 marks
        DISPLAY "Improving"
    ELSE IF Exam 3 marks < Exam 1 marks
        DISPLAY "Declining"
    ELSE
        DISPLAY "Unchanged"
    END IF

    IF all three marks are below 40
        DISPLAY "Consistently weak subject"
    END IF
END FOR

CALCULATE overall average for each examination
DISPLAY performance report
```

END IF

STOP

7.Python Source Code

```
number_of_subjects = int(input("Enter number of subjects: "))

subject_names = []
exam1_marks = []
exam2_marks = []
exam3_marks = []

total_exam1 = 0
total_exam2 = 0
total_exam3 = 0

valid_input = True

if number_of_subjects <= 0:
    print("Invalid number of subjects.")

else:
    for i in range(number_of_subjects):
        print("\nSubject", i + 1)

        subject = input("Enter subject name: ")

        mark1 = float(input("Enter marks in Examination 1: "))
        mark2 = float(input("Enter marks in Examination 2: "))
        mark3 = float(input("Enter marks in Examination 3: "))

        if (mark1 < 0 or mark1 > 100 or
            mark2 < 0 or mark2 > 100 or
            mark3 < 0 or mark3 > 100):

            print("Invalid marks. Marks must be between 0 and 100.")
            valid_input = False
            break

        subject_names.append(subject)
        exam1_marks.append(mark1)
        exam2_marks.append(mark2)
        exam3_marks.append(mark3)

        total_exam1 = total_exam1 + mark1
        total_exam2 = total_exam2 + mark2
        total_exam3 = total_exam3 + mark3

    if valid_input:
        print("\n--- STUDENT PERFORMANCE REPORT ---")

        for i in range(number_of_subjects):
            average = (
                exam1_marks[i] +
                exam2_marks[i] +
                exam3_marks[i]
            ) / 3
```

```
            print("\nSubject:", subject_names[i])
            print("Examination 1:", exam1_marks[i])
            print("Examination 2:", exam2_marks[i])
            print("Examination 3:", exam3_marks[i])
            print("Average marks:", round(average, 2))

            if exam3_marks[i] > exam1_marks[i]:
                print("Performance trend: Improving")

            elif exam3_marks[i] < exam1_marks[i]:
                print("Performance trend: Declining")

            else:
                print("Performance trend: Unchanged")

            if (exam1_marks[i] < 40 and
                exam2_marks[i] < 40 and
                exam3_marks[i] < 40):

                print("Warning: Consistently weak subject.")

        print("\n--- OVERALL EXAMINATION AVERAGES ---")

        print("Examination 1 average:",
              round(total_exam1 / number_of_subjects, 2))

        print("Examination 2 average:",
              round(total_exam2 / number_of_subjects, 2))

        print("Examination 3 average:",
              round(total_exam3 / number_of_subjects, 2))

        print("\nPerformance analysis completed.")
```

8. Test Cases

Test	Subject	Exam 1	Exam 2	Exam 3	Expected result
1	Mathematics	35	38	39	Improving; consistently weak
2	English	55	60	70	Improving
3	Science	80	75	72	Declining
4	Computer	65	65	65	Unchanged
5	History	45	50	105	Invalid marks
6	Any subject	—	—	—	Zero subjects should be rejected

9.Output screenshots of all the test cases

```
Enter number of subjects: 1

Subject 1
Enter subject name: Mathematics
Enter marks in Examination 1: 35
Enter marks in Examination 2: 38
Enter marks in Examination 3: 39

--- STUDENT PERFORMANCE REPORT ---

Subject: Mathematics
Examination 1: 35.0
Examination 2: 38.0
Examination 3: 39.0
Average marks: 37.33
Performance trend: Improving
Warning: Consistently weak subject.

--- OVERALL EXAMINATION AVERAGES ---
Examination 1 average: 35.0
Examination 2 average: 38.0
Examination 3 average: 39.0

Performance analysis completed.

================================================================================
Enter number of subjects: 1

Subject 1
Enter subject name: English
Enter marks in Examination 1: 55
Enter marks in Examination 2: 60
Enter marks in Examination 3: 70

--- STUDENT PERFORMANCE REPORT ---

Subject: English
Examination 1: 55.0
Examination 2: 60.0
Examination 3: 70.0
Average marks: 61.67
Performance trend: Improving

--- OVERALL EXAMINATION AVERAGES ---
Examination 1 average: 55.0
Examination 2 average: 60.0
Examination 3 average: 70.0

Performance analysis completed.
```

```
================================================================================
Enter number of subjects: 1

Subject 1
Enter subject name: Science
Enter marks in Examination 1: 80
Enter marks in Examination 2: 75
Enter marks in Examination 3: 72

--- STUDENT PERFORMANCE REPORT ---

Subject: Science
Examination 1: 80.0
Examination 2: 75.0
Examination 3: 72.0
Average marks: 75.67
Performance trend: Declining

--- OVERALL EXAMINATION AVERAGES ---
Examination 1 average: 80.0
Examination 2 average: 75.0
Examination 3 average: 72.0

Performance analysis completed.

================================================================================
Enter number of subjects: 1

Subject 1
Enter subject name: Computer
Enter marks in Examination 1: 65
Enter marks in Examination 2: 65
Enter marks in Examination 3: 65

--- STUDENT PERFORMANCE REPORT ---

Subject: Computer
Examination 1: 65.0
Examination 2: 65.0
Examination 3: 65.0
Average marks: 65.0
Performance trend: Unchanged

--- OVERALL EXAMINATION AVERAGES ---
Examination 1 average: 65.0
Examination 2 average: 65.0
Examination 3 average: 65.0

Performance analysis completed.

--------------------------------------------------------------------------------
```

10. Conclusion

The Student Performance Trend Analysis program demonstrates how Python can be used to record examination marks, calculate averages, compare results, and identify performance trends. It also highlights subjects where marks remain below the defined threshold across all three examinations.

This case study uses beginner-level Python concepts, including lists, loops, conditional statements, variables, arithmetic operations, and input/output. It can help students understand how simple programs can support academic performance monitoring.

Case Study 6: Traffic Violation Penalty Calculator

1. Problem Statement

Traffic violations can include overspeeding, driving without a helmet or seat belt, and other violations of traffic rules. Calculating penalties manually can be time-consuming and may lead to errors.

The objective of this case study is to develop a Python program that accepts traffic violation details, calculates the applicable penalty based on predefined rules, and displays a penalty summary for the driver.

2. Objectives

- To accept the number of traffic violations.
- To record the type of violation and relevant details.
- To calculate penalties using conditional statements.
- To calculate the total penalty amount.
- To classify violations according to their severity.
- To display a final traffic violation report.

3. Assumptions

1. The program uses a simplified set of penalty rules for educational purposes.
2. The penalty amounts below are assumed values for this program and are not official legal fines.
3. The program supports overspeeding, helmet violations, seat belt violations, and signal violations.
4. Overspeeding penalties depend on how far the driver exceeds the speed limit.
5. Each violation is processed separately.
6. The program calculates the total penalty for all entered violations.
7. Penalty amounts are expressed in rupees.

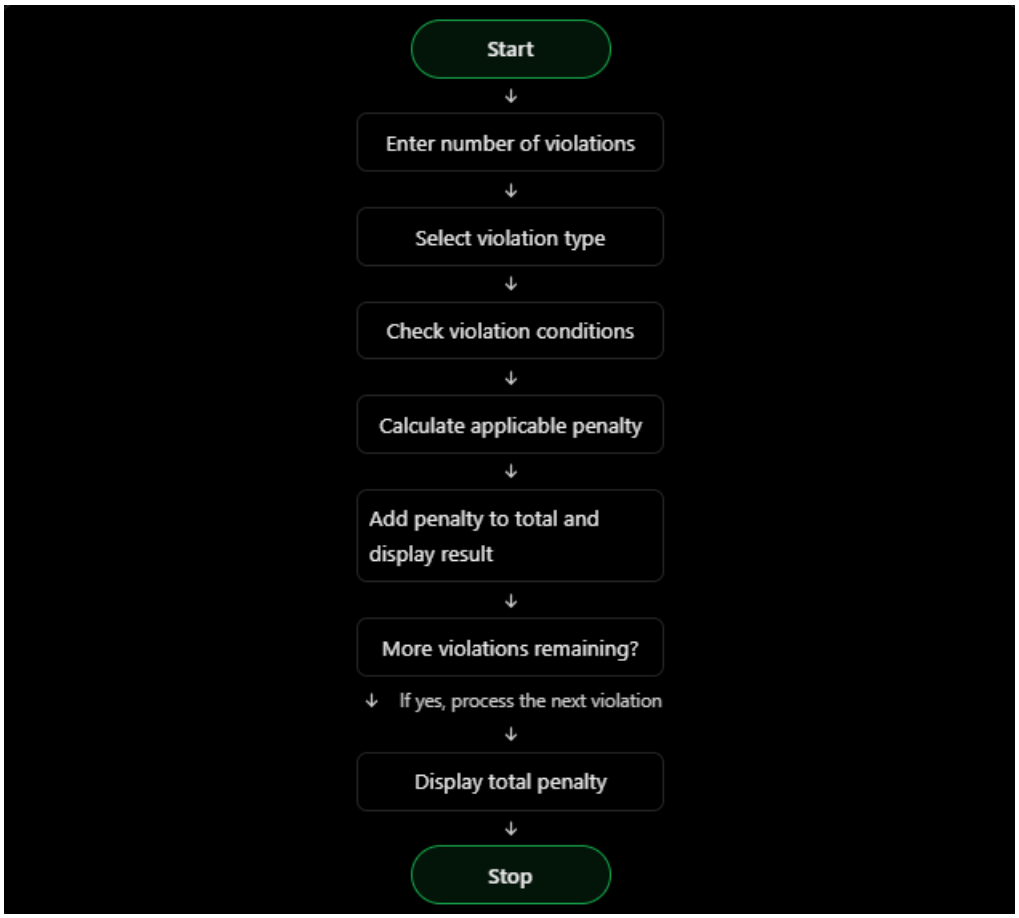
Assumed penalty rules

Violation	Condition	Penalty
Overspeeding	Up to 10 km/h over the limit	₹500
Overspeeding	More than 10 km/h over the limit	₹1,000
No helmet	Helmet not worn	₹500
No seat belt	Seat belt not worn	₹500
Signal violation	Traffic signal disobeyed	₹1,000

4. Algorithm

1. Start the program.
2. Input the number of violations.
3. Check whether the number of violations is valid.
4. Initialize the total penalty to zero.
5. Repeat for each violation:
    - Input the violation type.
    - If the violation is overspeeding, input the speed limit and vehicle speed.
    - Calculate the excess speed.
    - Apply the appropriate penalty.
    - For other violation types, assign the predefined penalty.
    - Add the penalty to the total.
    - Display the penalty for the violation.
6. Display the total penalty amount.
7. Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT number_of_violations
IF number_of_violations < 0
    DISPLAY "Invalid number of violations"
ELSE
    SET total_penalty = 0
    FOR each violation
        INPUT violation_type
        IF violation_type is "overspeeding"
            INPUT speed_limit
            INPUT vehicle_speed
            IF vehicle_speed <= speed_limit
```

```
            penalty = 0
        ELSE
            excess_speed = vehicle_speed - speed_limit
            IF excess_speed <= 10
                penalty = 500
            ELSE
                penalty = 1000
            END IF
        END IF
    ELSE IF violation_type is "helmet"
        penalty = 500
    ELSE IF violation_type is "seatbelt"
        penalty = 500
    ELSE IF violation_type is "signal"
        penalty = 1000
    ELSE
        DISPLAY "Invalid violation type"
        penalty = 0
    END IF
    total_penalty = total_penalty + penalty
    DISPLAY penalty
  END FOR
  DISPLAY total_penalty
END IF
STOP
```

7.Python Source Code

```
number_of_violations = int(input("Enter number of traffic violations: "))

total_penalty = 0
valid_violations = 0
invalid_violations = 0

if number_of_violations < 0:
    print("Invalid number of violations.")

else:
    for i in range(number_of_violations):
        print("\nViolation", i + 1)

        print("1. Overspeeding")
        print("2. No helmet")
        print("3. No seat belt")
        print("4. Signal violation")

        choice = input("Enter violation type (1-4): ")

        penalty = 0
        violation_name = ""

        if choice == "1":
            violation_name = "Overspeeding"

            speed_limit = float(input("Enter speed limit (km/h): "))
            vehicle_speed = float(input("Enter vehicle speed (km/h): "))

            if speed_limit < 0 or vehicle_speed < 0:
                print("Invalid speed entered.")
                invalid_violations = invalid_violations + 1

            elif vehicle_speed <= speed_limit:
                penalty = 0
                valid_violations = valid_violations + 1

            else:
                excess_speed = vehicle_speed - speed_limit

                if excess_speed <= 10:
                    penalty = 500
                else:
                    penalty = 1000

                valid_violations = valid_violations + 1

        elif choice == "2":
            violation_name = "No helmet"
            penalty = 500
            valid_violations = valid_violations + 1

        elif choice == "3":
            violation_name = "No seat belt"
            penalty = 500
            valid_violations = valid_violations + 1
```

```
        elif choice == "4":
            violation_name = "Signal violation"
            penalty = 1000
            valid_violations = valid_violations + 1

        else:
            print("Invalid violation type.")
            invalid_violations = invalid_violations + 1

        if choice in ["1", "2", "3", "4"]:
            if not (choice == "1" and
                    (speed_limit < 0 or vehicle_speed < 0)):
                print("Violation:", violation_name)
                print("Penalty: Rs.", penalty)
                total_penalty = total_penalty + penalty

    print("\n--- TRAFFIC VIOLATION REPORT ---")
    print("Valid violations:", valid_violations)
    print("Invalid entries:", invalid_violations)
    print("Total penalty: Rs.", total_penalty)
    print("Report completed.")
```

8. Test Cases

	Test Input	Expected result
1	Overspeeding: limit 60, speed 65	₹500 penalty
2	Overspeeding: limit 60, speed 75	₹1,000 penalty
3	No helmet	₹500 penalty
4	No seat belt	₹500 penalty
5	Signal violation	₹1,000 penalty
6	Overspeeding: limit 60, speed 55	₹0 penalty
7	Invalid violation type	Invalid entry recorded
8	Negative number of violations	Invalid input message

9.Output Screenshots for all the test cases

```
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 1
Enter speed limit (km/h): 60
Enter vehicle speed (km/h): 65
Violation: Overspeeding
Penalty: Rs. 500

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 500
Report completed.

================================================================================
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 1
Enter speed limit (km/h): 60
Enter vehicle speed (km/h): 75
Violation: Overspeeding
Penalty: Rs. 1000

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 1000
Report completed.

--------------------------------------------------------------------------------
```

```
--------------------------------------------------------------------------------------------
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 2
Violation: No helmet
Penalty: Rs. 500

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 500
Report completed.

============================================================================================
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 3
Violation: No seat belt
Penalty: Rs. 500

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 500
Report completed.

============================================================================================
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 4
Violation: Signal violation
Penalty: Rs. 1000

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 1000
Report completed.

--------------------------------------------------------------------------------------------
```

```
Enter number of traffic violations: 1

Violation 1
1. Overspeeding
2. No helmet
3. No seat belt
4. Signal violation
Enter violation type (1-4): 1
Enter speed limit (km/h): 60
Enter vehicle speed (km/h): 55
Violation: Overspeeding
Penalty: Rs. 0

--- TRAFFIC VIOLATION REPORT ---
Valid violations: 1
Invalid entries: 0
Total penalty: Rs. 0
Report completed.
```

10.Conclusion

The Traffic Violation Penalty Calculator demonstrates how Python can automate the calculation of penalties for different traffic violations. It uses conditional statements to select the appropriate penalty, loops to process multiple violations, and variables to calculate the total amount.

This case study helps beginners practise input/output, arithmetic operations, loops, and if-elif-else conditions. The penalty values are illustrative assumptions for the assignment, not official traffic fines.

Case Study 7: Progressive Water Tariff Analysis

1. Problem Statement

Water consumption charges can be calculated using a progressive tariff system, in which different portions of water usage are charged at different rates. A program can help calculate the total water bill based on the quantity consumed and the applicable tariff slabs.

The aim of this case study is to develop a Python program that accepts water consumption, calculates the bill using progressive rates, and displays a detailed breakdown of the charges.

2. Objectives

- To accept the total water consumption in kilolitres (kL).
- To apply different rates to different consumption slabs.
- To calculate the total water bill.
- To display the charge for each applicable slab.
- To validate the consumption input.
- To generate a clear water bill summary.

3. Assumptions

The following tariff rates are illustrative values for this assignment, not official water utility rates.

Consumption slab	Assumed rate
First 10 kL	₹5 per kL
Next 10 kL (11–20 kL)	₹8 per kL
Above 20 kL	₹12 per kL

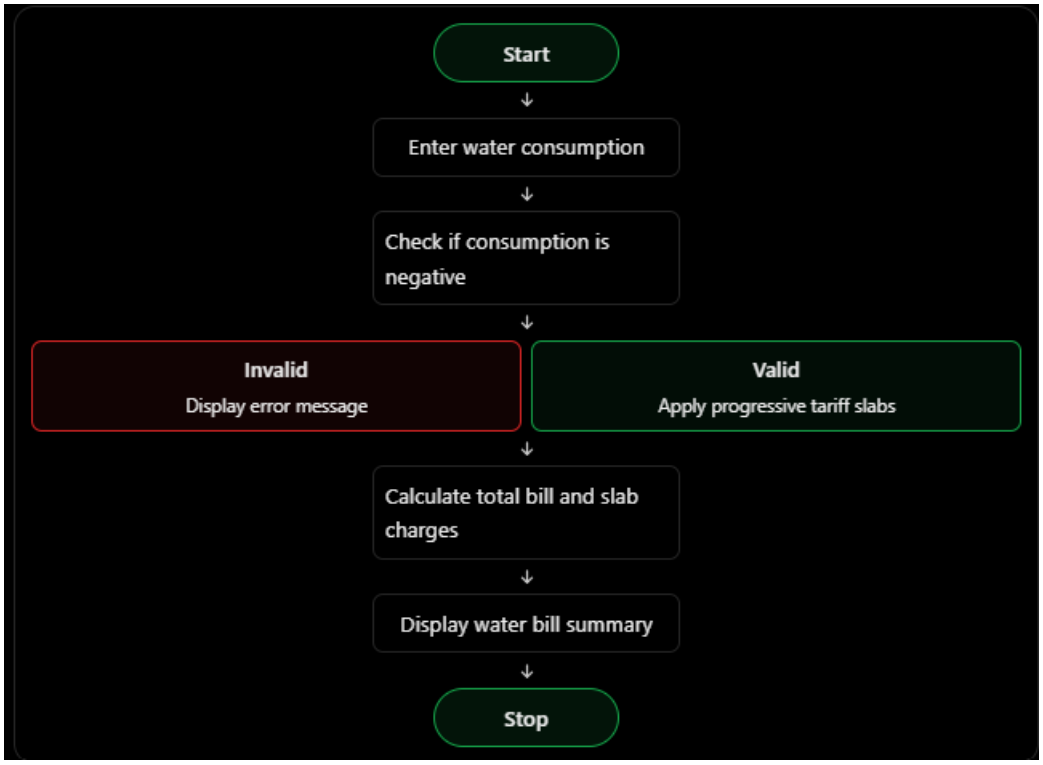
Additional assumptions:

1. Water consumption is entered in kilolitres.
2. Consumption cannot be negative.
3. Each rate applies only to the portion of consumption within its slab.
4. The bill is calculated by adding the charges from all applicable slabs.
5. No fixed charges, taxes, or late fees are included.

4. Algorithm

1. Start the program.
2. Input the water consumption in kilolitres.
3. Check whether the consumption is negative.
4. If consumption is invalid, display an error message.
5. Otherwise, calculate the charge:
    - If consumption is up to 10 kL, charge ₹5 per kL.
    - If consumption is between 10 and 20 kL, charge ₹5 for the first 10 kL and ₹8 for the remaining consumption.
    - If consumption is above 20 kL, charge ₹5 for the first 10 kL, ₹8 for the next 10 kL, and ₹12 for the remaining consumption.
6. Calculate the total bill.
7. Display consumption, slab-wise charges, and total bill.
8. Stop the program.

5.Flow Chart



6.Pseudo Code

```
START
INPUT water_consumption
IF water_consumption < 0
   DISPLAY "Invalid water consumption"
ELSE
   IF water_consumption <= 10
      slab1_charge = water_consumption * 5
      slab2_charge = 0
      slab3_charge = 0
   ELSE IF water_consumption <= 20
      slab1_charge = 10 * 5
      slab2_charge = (water_consumption - 10) * 8
      slab3_charge = 0
   ELSE
      slab1_charge = 10 * 5
      slab2_charge = 10 * 8
      slab3_charge = (water_consumption - 20) * 12
   END IF
   total_bill = slab1_charge + slab2_charge + slab3_charge
   DISPLAY slab1_charge
   DISPLAY slab2_charge
   DISPLAY slab3_charge
   DISPLAY total_bill
END IF
STOP
```

7.Python Source Code

```
water_consumption = float(
    input("Enter water consumption in kL: ")
)

slab1_charge = 0
slab2_charge = 0
slab3_charge = 0

if water_consumption < 0:
    print("Invalid water consumption.")
    print("Consumption cannot be negative.")

else:
    if water_consumption <= 10:
        slab1_charge = water_consumption * 5

    elif water_consumption <= 20:
        slab1_charge = 10 * 5
        slab2_charge = (water_consumption - 10) * 8

    else:
        slab1_charge = 10 * 5
        slab2_charge = 10 * 8
        slab3_charge = (water_consumption - 20) * 12

    total_bill = slab1_charge + slab2_charge + slab3_charge

    print("\n--- WATER BILL SUMMARY ---")
    print("Water consumption:", water_consumption, "kL")

    print("First 10 kL charge: Rs.",
          round(slab1_charge, 2))

    print("Next 10 kL charge: Rs.",
          round(slab2_charge, 2))

    print("Above 20 kL charge: Rs.",
          round(slab3_charge, 2))

    print("Total water bill: Rs.",
          round(total_bill, 2))
```

8. Test Cases

Test	Water consumption	Expected slab charges	Expected total
1	5 kL	₹25 + ₹0 + ₹0	₹25
2	10 kL	₹50 + ₹0 + ₹0	₹50
3	15 kL	₹50 + ₹40 + ₹0	₹90
4	20 kL	₹50 + ₹80 + ₹0	₹130
5	25 kL	₹50 + ₹80 + ₹60	₹190
6	0 kL	₹0 + ₹0 + ₹0	₹0
7	-5 kL	Invalid consumption	Error message

9.Outputs for all the test cases

```
Enter water consumption in kL: 5

--- WATER BILL SUMMARY ---
Water consumption: 5.0 kL
First 10 kL charge: Rs. 25.0
Next 10 kL charge: Rs. 0
Above 20 kL charge: Rs. 0
Total water bill: Rs. 25.0

================================================================================
Enter water consumption in kL: 10

--- WATER BILL SUMMARY ---
Water consumption: 10.0 kL
First 10 kL charge: Rs. 50.0
Next 10 kL charge: Rs. 0
Above 20 kL charge: Rs. 0
Total water bill: Rs. 50.0

================================================================================
Enter water consumption in kL: 15

--- WATER BILL SUMMARY ---
Water consumption: 15.0 kL
First 10 kL charge: Rs. 50
Next 10 kL charge: Rs. 40.0
Above 20 kL charge: Rs. 0
Total water bill: Rs. 90.0

================================================================================
Enter water consumption in kL: 20

--- WATER BILL SUMMARY ---
Water consumption: 20.0 kL
First 10 kL charge: Rs. 50
Next 10 kL charge: Rs. 80.0
Above 20 kL charge: Rs. 0
Total water bill: Rs. 130.0

================================================================================
Enter water consumption in kL: 25

--- WATER BILL SUMMARY ---
Water consumption: 25.0 kL
First 10 kL charge: Rs. 50
Next 10 kL charge: Rs. 80
Above 20 kL charge: Rs. 60.0
Total water bill: Rs. 190.0

================================================================================

================================================================================
Enter water consumption in kL: 0

--- WATER BILL SUMMARY ---
Water consumption: 0.0 kL
First 10 kL charge: Rs. 0.0
Next 10 kL charge: Rs. 0
Above 20 kL charge: Rs. 0
Total water bill: Rs. 0.0

================================================================================
Enter water consumption in kL: -5
Invalid water consumption.
Consumption cannot be negative.
```

10. Conclusion

The Progressive Water Tariff Analysis program calculates water charges using a tiered pricing system. It applies different rates to different portions of water consumption and displays the individual slab charges along with the total bill.

This case study helps beginners understand conditional statements, arithmetic operations, variables, input/output, and validation in Python. It also demonstrates how programming can be used to automate utility bill calculations.

Case Study 8: Food Waste Monitoring System

1. Problem Statement

Food waste is a common problem in canteens, restaurants, hostels, and other food-serving establishments. Monitoring the quantity of food prepared, served, and wasted can help identify waste patterns and improve food management.

The objective of this case study is to develop a Python program that records food preparation and waste quantities, calculates the percentage of food wasted, and displays a summary that helps users monitor food waste.

2. Objectives

- To record the quantity of food prepared each day.
- To record the quantity of food served and wasted.
- To calculate the quantity of food remaining.
- To calculate the percentage of food wasted.
- To classify food waste according to a defined threshold.
- To display a daily food waste report.

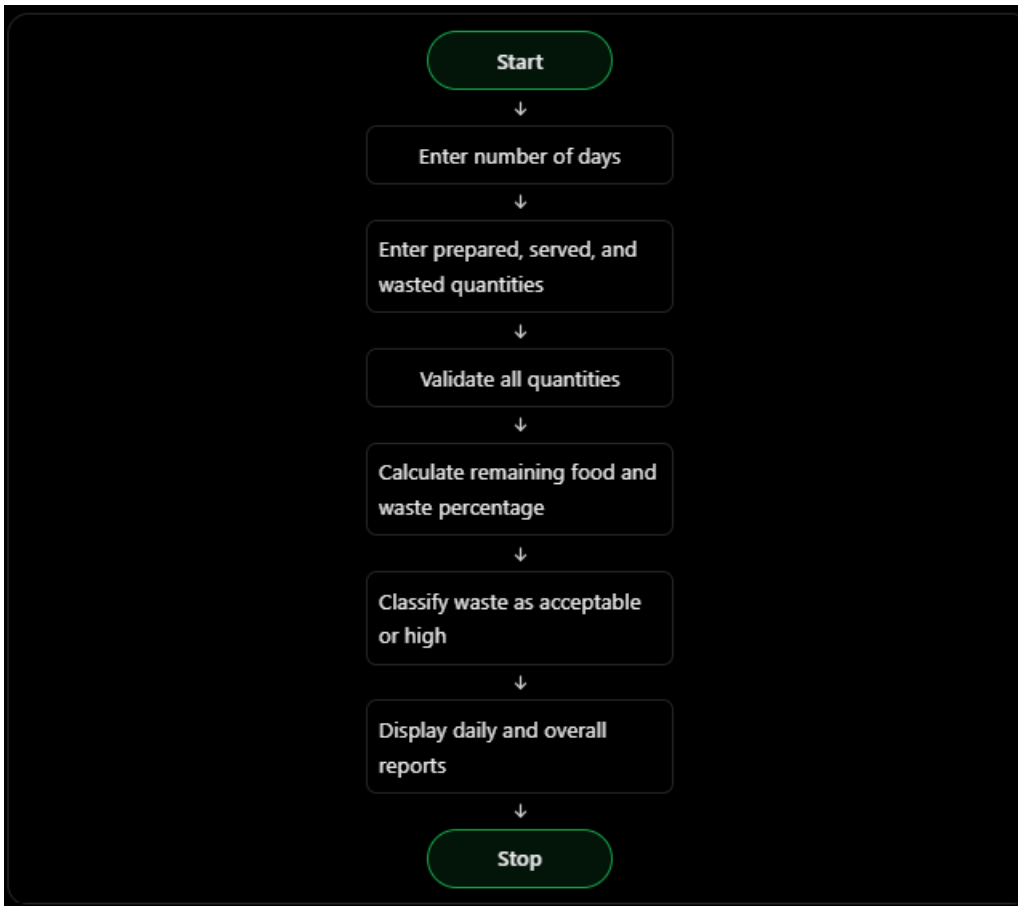
3. Assumptions

1. Food quantities are measured in kilograms (kg).
2. Food prepared, food served, and food wasted must be non-negative.
3. The total quantity served and wasted cannot exceed the quantity prepared.
4. Food waste percentage is calculated using the quantity prepared.
5. A waste percentage above 10% is considered high for this educational example.
6. The program can monitor multiple days.
7. The waste threshold is an illustrative assumption, not an official standard.

4. Algorithm

1. Start the program.
2. Input the number of days to monitor.
3. Check whether the number of days is valid.
4. Initialize variables for total food prepared, served, and wasted.
5. For each day:
    - Input the quantity of food prepared.
    - Input the quantity of food served.
    - Input the quantity of food wasted.
    - Validate the quantities.
    - Calculate the remaining food.
    - Calculate the waste percentage.
    - Classify the waste level.
    - Display the daily report.
6. Calculate the average food waste percentage across all monitored days.
7. Display the overall summary.
8. Stop the program.

5.Flowchart



6.Pseudo Code

```
START
INPUT number_of_days
IF number_of_days <= 0
    DISPLAY "Invalid number of days"
ELSE
    SET total_prepared = 0
    SET total_served = 0
    SET total_wasted = 0
    FOR each day
        INPUT food_prepared
        INPUT food_served
```

```
        INPUT food_wasted
        IF any quantity is negative
            DISPLAY "Invalid quantity"
        ELSE IF food_served + food_wasted > food_prepared
            DISPLAY "Quantities exceed food prepared"
        ELSE
            remaining_food = food_prepared - food_served - food_wasted
            IF food_prepared > 0
                waste_percentage = food_wasted / food_prepared * 100
            ELSE
                waste_percentage = 0
            END IF
            IF waste_percentage > 10
                DISPLAY "High food waste"
            ELSE
                DISPLAY "Acceptable food waste"
            END IF
            UPDATE total quantities
            DISPLAY daily report
        END IF
    END FOR
    DISPLAY overall food waste summary
END IF
STOP
```

7.Python Source Code

```
number_of_days = int(input("Enter number of days to monitor: "))

total_prepared = 0
total_served = 0
total_wasted = 0
valid_days = 0

if number_of_days <= 0:
    print("Invalid number of days.")

else:
    for i in range(number_of_days):
        print("\nDay", i + 1)

        food_prepared = float(
            input("Enter food prepared (kg): ")
        )

        food_served = float(
            input("Enter food served (kg): ")
        )

        food_wasted = float(
            input("Enter food wasted (kg): ")
        )

        if (food_prepared < 0 or
            food_served < 0 or
            food_wasted < 0):

            print("Invalid quantity. Values cannot be negative.")

        elif food_served + food_wasted > food_prepared:
            print("Invalid data. Served and wasted food cannot")
            print("exceed the quantity prepared.")

        else:
            remaining_food = (
                food_prepared - food_served - food_wasted
            )

            if food_prepared > 0:
                waste_percentage = (
                    food_wasted / food_prepared
                ) * 100
            else:
                waste_percentage = 0

            print("\n--- DAILY FOOD WASTE REPORT ---")
            print("Food prepared:", food_prepared, "kg")
            print("Food served:", food_served, "kg")
            print("Food wasted:", food_wasted, "kg")
            print("Remaining food:", remaining_food, "kg")
            print("Waste percentage:",
                  round(waste_percentage, 2), "%")
```

```
            if waste_percentage > 10:
                print("Waste status: High food waste")
            else:
                print("Waste status: Acceptable food waste")

            total_prepared = total_prepared + food_prepared
            total_served = total_served + food_served
            total_wasted = total_wasted + food_wasted
            valid_days = valid_days + 1

    print("\n--- OVERALL FOOD WASTE SUMMARY ---")
    print("Valid days recorded:", valid_days)
    print("Total food prepared:", total_prepared, "kg")
    print("Total food served:", total_served, "kg")
    print("Total food wasted:", total_wasted, "kg")

    if total_prepared > 0:
        overall_waste_percentage = (
            total_wasted / total_prepared
        ) * 100

        print("Overall waste percentage:",
              round(overall_waste_percentage, 2), "%")
    else:
        print("Overall waste percentage: 0.0%")

    print("Monitoring completed.")
```

8. Test Cases

Test	Prepared	Served	Wasted	Expected result
1	100 kg	80 kg	10 kg	10% waste; acceptable
2	100 kg	75 kg	15 kg	15% waste; high waste
3	50 kg	40 kg	5 kg	10% waste; acceptable
4	0 kg	0 kg	0 kg	0% waste
5	100 kg	80 kg	25 kg	Invalid; quantities exceed prepared food
6	-10 kg	5 kg	2 kg	Invalid quantity
7	Any quantities	—	—	Zero monitoring days should be rejected

9.Output screenshots for all the test cases

```
Enter number of days to monitor: 1

Day 1
Enter food prepared (kg): 100
Enter food served (kg): 80
Enter food wasted (kg): 10

--- DAILY FOOD WASTE REPORT ---
Food prepared: 100.0 kg
Food served: 80.0 kg
Food wasted: 10.0 kg
Remaining food: 10.0 kg
Waste percentage: 10.0 %
Waste status: Acceptable food waste

--- OVERALL FOOD WASTE SUMMARY ---
Valid days recorded: 1
Total food prepared: 100.0 kg
Total food served: 80.0 kg
Total food wasted: 10.0 kg
Overall waste percentage: 10.0 %
Monitoring completed.
```

```
Enter number of days to monitor: 1

Day 1
Enter food prepared (kg): 100
Enter food served (kg): 75
Enter food wasted (kg): 15

--- DAILY FOOD WASTE REPORT ---
Food prepared: 100.0 kg
Food served: 75.0 kg
Food wasted: 15.0 kg
Remaining food: 10.0 kg
Waste percentage: 15.0 %
Waste status: High food waste

--- OVERALL FOOD WASTE SUMMARY ---
Valid days recorded: 1
Total food prepared: 100.0 kg
Total food served: 75.0 kg
Total food wasted: 15.0 kg
Overall waste percentage: 15.0 %
Monitoring completed.
```

```
==================================================
Enter number of days to monitor: 1

Day 1
Enter food prepared (kg): 50
Enter food served (kg): 40
Enter food wasted (kg): 5

--- DAILY FOOD WASTE REPORT ---
Food prepared: 50.0 kg
Food served: 40.0 kg
Food wasted: 5.0 kg
Remaining food: 5.0 kg
Waste percentage: 10.0 %
Waste status: Acceptable food waste

--- OVERALL FOOD WASTE SUMMARY ---
Valid days recorded: 1
Total food prepared: 50.0 kg
Total food served: 40.0 kg
Total food wasted: 5.0 kg
Overall waste percentage: 10.0 %
Monitoring completed.
```

10.Conclusion

The Food Waste Monitoring System demonstrates how Python can be used to record food quantities, calculate waste percentages, validate data, and identify days with high food waste.

This case study uses beginner-level concepts such as variables, loops, conditional statements, arithmetic operations, and input/output. The program can help canteens and food-service establishments monitor waste and make more informed decisions about food preparation.

Case Study 9: Smart Canteen Order and Stock Management

1. Problem Statement

A canteen needs to manage food orders, calculate order costs, and monitor the availability of food items. Manual order processing can lead to calculation errors and difficulties in tracking stock.

The aim of this case study is to develop a Python program that accepts canteen orders, checks item availability, calculates the total bill, updates stock quantities, and displays an order summary.

2. Objectives

- To maintain a list of food items and their prices.
- To store the available stock of each food item.
- To accept customer orders and quantities.
- To check stock availability before confirming an order.
- To calculate the total cost of the order.
- To update stock after a successful order.
- To display the final bill and remaining stock.

3. Assumptions

1. The canteen offers a fixed menu of food items.
2. Prices and stock quantities are predefined in the program.
3. Customers select items using menu numbers.
4. Order quantities must be positive integers.
5. An order is accepted only if sufficient stock is available.
6. Stock is reduced only when an order is successfully processed.
7. The program uses sample prices and quantities for educational purposes.

Sample canteen menu

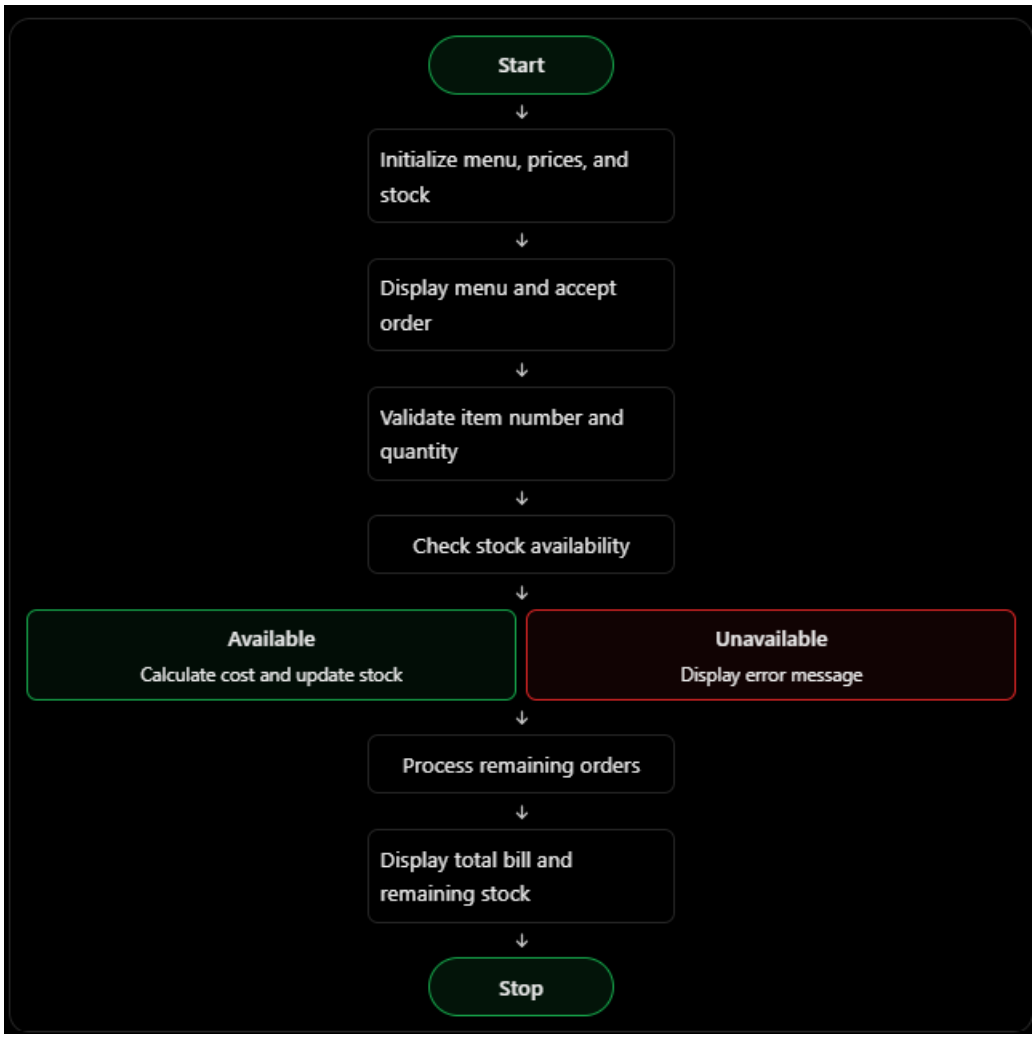
Menu number	Food item	Price	Initial stock
1	Sandwich	₹50	10
2	Veg Burger	₹80	8

3	Samosa	₹20	20
4	Juice	₹30	15

4. Algorithm

1. Start the program.
2. Create lists for food names, prices, and available stock.
3. Input the number of orders to process.
4. Validate the number of orders.
5. For each order:
    - Display the menu.
    - Input the menu number.
    - Input the quantity required.
    - Check whether the menu number is valid.
    - Check whether the quantity is positive.
    - Check whether sufficient stock is available.
    - If valid, calculate the cost and update the stock.
    - Otherwise, display an appropriate error message.
6. Calculate the total amount for all successful orders.
7. Display the bill and updated stock.
8. Stop the program.

5.Flow Chart



6.Pseudocode

```
START
CREATE lists for food names, prices, and stock
SET total_bill = 0
INPUT number_of_orders
IF number_of_orders < 0
    DISPLAY "Invalid number of orders"
ELSE
    FOR each order
        DISPLAY menu
        INPUT item_number
```

```
        INPUT quantity
        IF item_number is invalid
            DISPLAY "Invalid menu selection"
        ELSE IF quantity <= 0
            DISPLAY "Invalid quantity"
        ELSE IF quantity > available stock
            DISPLAY "Insufficient stock"
        ELSE
            cost = price * quantity
            total_bill = total_bill + cost
            UPDATE stock
            DISPLAY order confirmation
        END IF
    END FOR
    DISPLAY total bill
    DISPLAY updated stock
END IF
STOP
```

7.Python Source Code

```
food_items = ["Sandwich", "Veg Burger", "Samosa", "Juice"]
prices = [50, 80, 20, 30]
stock = [10, 8, 20, 15]

total_bill = 0
successful_orders = 0
rejected_orders = 0

number_of_orders = int(input("Enter number of orders: "))

if number_of_orders < 0:
    print("Invalid number of orders.")

else:
    for i in range(number_of_orders):
        print("\n--- CANTEEN MENU ---")

        for j in range(len(food_items)):
            print(j + 1, ".", food_items[j],
                  "- Rs.", prices[j],
                  "- Stock:", stock[j])

        item_number = int(input("Enter menu number: "))
        quantity = int(input("Enter quantity: "))

        if item_number < 1 or item_number > len(food_items):
            print("Invalid menu selection.")
            rejected_orders = rejected_orders + 1

        elif quantity <= 0:
            print("Invalid quantity.")
            rejected_orders = rejected_orders + 1

        else:
            item_index = item_number - 1

            if quantity > stock[item_index]:
                print("Insufficient stock for",
                      food_items[item_index])
                print("Available stock:", stock[item_index])
                rejected_orders = rejected_orders + 1

            else:
                cost = prices[item_index] * quantity

                stock[item_index] = (
                    stock[item_index] - quantity
                )

                total_bill = total_bill + cost
                successful_orders = successful_orders + 1

                print("Order successful.")
                print("Item:", food_items[item_index])
                print("Quantity:", quantity)
                print("Order cost: Rs.", cost)
                print("Remaining stock:", stock[item_index])
```

```
    print("\n--- CANTEEN BILL SUMMARY ---")
    print("Successful orders:", successful_orders)
    print("Rejected orders:", rejected_orders)
    print("Total bill: Rs.", total_bill)

    print("\n--- UPDATED STOCK REPORT ---")

    for i in range(len(food_items)):
        print(food_items[i], ":", stock[i], "units remaining")

    print("\nOrder processing completed.")
```

8. Test Cases

	Test Input	Expected result
1	Sandwich, quantity 2	Order accepted; bill ₹100
2	Veg Burger, quantity 3	Order accepted; bill ₹240
3	Samosa, quantity 5	Order accepted; bill ₹100
4	Juice, quantity 2	Order accepted; bill ₹60
5	Sandwich, quantity 15	Rejected due to insufficient stock
6	Invalid menu number 8	Invalid selection message
7	Quantity 0	Invalid quantity message
8	Negative number of orders	Invalid input message

9.Output Screenshots for all the test cases

```
Enter number of orders: 1

--- CANTEEN MENU ---
1 . Sandwich - Rs. 50 - Stock: 10
2 . Veg Burger - Rs. 80 - Stock: 8
3 . Samosa - Rs. 20 - Stock: 20
4 . Juice - Rs. 30 - Stock: 15
Enter menu number: 1
Enter quantity: 2
Order successful.
Item: Sandwich
Quantity: 2
Order cost: Rs. 100
Remaining stock: 8

--- CANTEEN BILL SUMMARY ---
Successful orders: 1
Rejected orders: 0
Total bill: Rs. 100

--- UPDATED STOCK REPORT ---
Sandwich : 8 units remaining
Veg Burger : 8 units remaining
Samosa : 20 units remaining
Juice : 15 units remaining

Order processing completed.

================================================================
```

```
================================================================================================
Enter number of orders: 1

--- CANTEEN MENU ---
1 . Sandwich - Rs. 50 - Stock: 10
2 . Veg Burger - Rs. 80 - Stock: 8
3 . Samosa - Rs. 20 - Stock: 20
4 . Juice - Rs. 30 - Stock: 15
Enter menu number: 2
Enter quantity: 3
Order successful.
Item: Veg Burger
Quantity: 3
Order cost: Rs. 240
Remaining stock: 5

--- CANTEEN BILL SUMMARY ---
Successful orders: 1
Rejected orders: 0
Total bill: Rs. 240

--- UPDATED STOCK REPORT ---
Sandwich : 10 units remaining
Veg Burger : 5 units remaining
Samosa : 20 units remaining
Juice : 15 units remaining

Order processing completed.

================================================================================================
Enter number of orders: 1

--- CANTEEN MENU ---
1 . Sandwich - Rs. 50 - Stock: 10
2 . Veg Burger - Rs. 80 - Stock: 8
3 . Samosa - Rs. 20 - Stock: 20
4 . Juice - Rs. 30 - Stock: 15
Enter menu number: 3
Enter quantity: 5
Order successful.
Item: Samosa
Quantity: 5
Order cost: Rs. 100
Remaining stock: 15

--- CANTEEN BILL SUMMARY ---
Successful orders: 1
Rejected orders: 0
Total bill: Rs. 100

--- UPDATED STOCK REPORT ---
Sandwich : 10 units remaining
Veg Burger : 8 units remaining
Samosa : 15 units remaining
Juice : 15 units remaining

Order processing completed.
```

10. Conclusion

The Smart Canteen Order and Stock Management program demonstrates how Python can be used to process customer orders, calculate bills, validate quantities, and update inventory automatically.

The program uses lists, loops, conditional statements, variables, and arithmetic operations. It also demonstrates how stock availability can be checked before confirming an order, helping prevent orders that exceed available inventory.

Case Study 10: Weekly Electricity Consumption Matrix

1. Problem Statement

Electricity consumption varies from day to day depending on the usage of electrical appliances. Monitoring daily electricity consumption helps identify high-usage days, calculate total consumption, and understand electricity usage patterns.

The objective of this case study is to develop a Python program that records electricity consumption for seven days, calculates daily and weekly totals, identifies the day with the highest consumption, and displays a weekly electricity consumption report.

2. Objectives

- To record electricity consumption for seven days.
- To store daily readings using a Python list.
- To calculate total weekly electricity consumption.
- To calculate average daily consumption.
- To identify the day with the highest electricity consumption.
- To identify days when consumption exceeds a specified threshold.
- To display a weekly electricity consumption report.

3. Assumptions

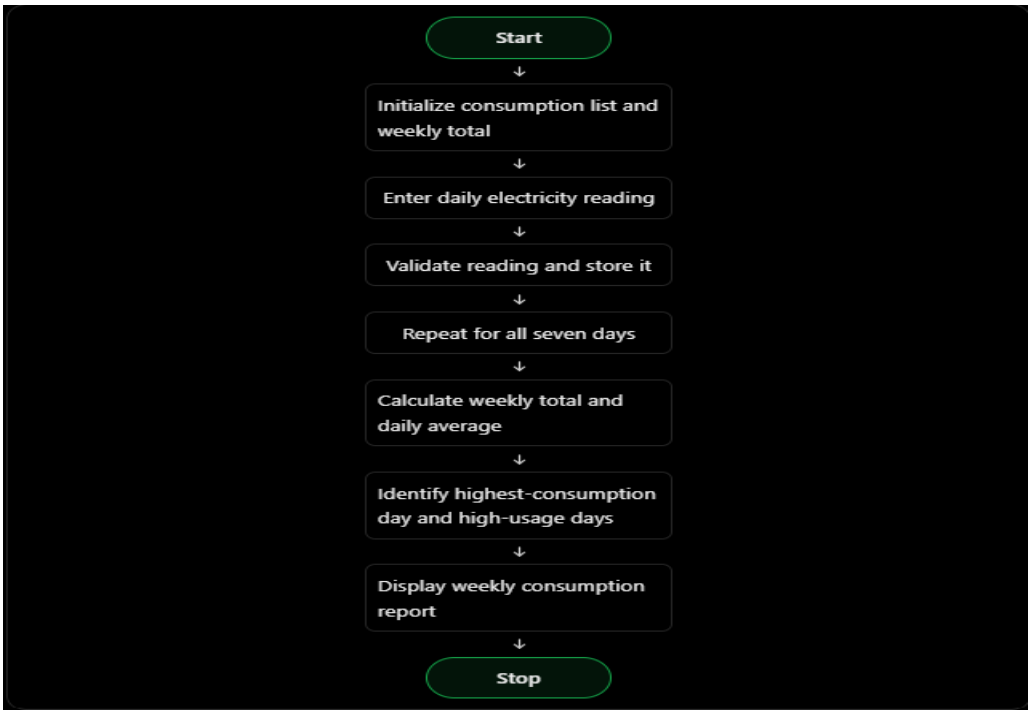
1. Electricity consumption is measured in kilowatt-hours (kWh).
2. The program records consumption for seven consecutive days.
3. Consumption values cannot be negative.
4. The daily high-consumption threshold is assumed to be 10 kWh for this educational example.
5. Weekly consumption is the sum of the seven daily readings.
6. Average daily consumption is calculated by dividing the weekly total by seven.

7. The program analyzes consumption only and does not calculate electricity charges.

4. Algorithm

1. Start the program.
2. Create a list to store daily electricity consumption.
3. Initialize the weekly total to zero.
4. Repeat the following steps for seven days:
    - Input the electricity consumption for the day.
    - Check whether the reading is negative.
    - If valid, add the reading to the list.
    - Add the reading to the weekly total.
5. If an invalid reading is entered, display an error message.
6. Calculate the average daily consumption.
7. Identify the day with the highest consumption.
8. Compare each day's consumption with the threshold of 10 kWh.
9. Display daily readings, weekly total, average consumption, and the highest-consumption day.
10. Stop the program.

5.Flow Chart



6.Pseudo Code

```
START
CREATE empty list daily_consumption
SET weekly_total = 0
FOR each of the 7 days
    INPUT daily_reading
    IF daily_reading < 0
        DISPLAY "Invalid electricity reading"
        STOP
    ELSE
        ADD daily_reading to daily_consumption
        ADD daily_reading to weekly_total
    END IF
END FOR
CALCULATE average_consumption = weekly_total / 7
SET highest_consumption = first daily reading
```

```
SET highest_day = first day
FOR each day
    IF daily reading > highest_consumption
        UPDATE highest_consumption
        UPDATE highest_day
    END IF
END FOR
DISPLAY daily consumption readings
DISPLAY weekly total
DISPLAY average daily consumption
DISPLAY highest-consumption day
FOR each day
    IF daily reading > 10
        DISPLAY "High consumption"
    ELSE
        DISPLAY "Normal consumption"
    END IF
END FOR
STOP
```

7.Python Source Code

```
days = ["Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday", "Sunday"]

daily_consumption = []
weekly_total = 0
valid_input = True

threshold = 10

print("--- WEEKLY ELECTRICITY CONSUMPTION ---")

for i in range(7):
    reading = float(
        input("Enter electricity consumption for " +
              days[i] + " (kWh): ")
    )

    if reading < 0:
        print("Invalid reading. Consumption cannot be negative.")
        valid_input = False
        break

    daily_consumption.append(reading)
    weekly_total = weekly_total + reading

if valid_input:
    average_consumption = weekly_total / 7

    highest_consumption = daily_consumption[0]
    highest_day = days[0]

    for i in range(7):
        if daily_consumption[i] > highest_consumption:
            highest_consumption = daily_consumption[i]
            highest_day = days[i]

    print("\n--- DAILY CONSUMPTION REPORT ---")

    for i in range(7):
        print(days[i], ":", daily_consumption[i], "kWh")

        if daily_consumption[i] > threshold:
            print("  Status: High consumption")
        else:
            print("  Status: Normal consumption")

    print("\n--- WEEKLY SUMMARY ---")
    print("Total weekly consumption:",
          round(weekly_total, 2), "kWh")

    print("Average daily consumption:",
          round(average_consumption, 2), "kWh")

    print("Highest consumption:",
          highest_consumption, "kWh")

    print("Day with highest consumption:", highest_day)

    high_days = 0

    for i in range(7):
        if daily_consumption[i] > threshold:
            high_days = high_days + 1

    print("Number of high-consumption days:", high_days)
    print("Weekly analysis completed.")
```

8. Test Cases

Test	Daily readings (kWh)	Expected result
1	5, 6, 7, 8, 9, 10, 5	Total: 50 kWh; average: 7.14 kWh
2	12, 15, 11, 9, 8, 14, 10	Total: 79 kWh; highest: 15 kWh on Tuesday
3	0, 0, 0, 0, 0, 0, 0	Total and average are zero
4	5, 7, -2, 8, 6, 5, 4	Invalid reading detected
5	10, 10, 10, 10, 10, 10, 10	Total: 70 kWh; average: 10 kWh

9.Output Screenshots for the test cases

```
--- WEEKLY ELECTRICITY CONSUMPTION ---
Enter electricity consumption for Monday (kWh): 5
Enter electricity consumption for Tuesday (kWh): 6
Enter electricity consumption for Wednesday (kWh): 7
Enter electricity consumption for Thursday (kWh): 8
Enter electricity consumption for Friday (kWh): 9
Enter electricity consumption for Saturday (kWh): 10
Enter electricity consumption for Sunday (kWh): 5

--- DAILY CONSUMPTION REPORT ---
Monday : 5.0 kWh
  Status: Normal consumption
Tuesday : 6.0 kWh
  Status: Normal consumption
Wednesday : 7.0 kWh
  Status: Normal consumption
Thursday : 8.0 kWh
  Status: Normal consumption
Friday : 9.0 kWh
  Status: Normal consumption
Saturday : 10.0 kWh
  Status: Normal consumption
Sunday : 5.0 kWh
  Status: Normal consumption

--- WEEKLY SUMMARY ---
Total weekly consumption: 50.0 kWh
Average daily consumption: 7.14 kWh
Highest consumption: 10.0 kWh
Day with highest consumption: Saturday
Number of high-consumption days: 0
Weekly analysis completed.
```

```
================================================================================
--- WEEKLY ELECTRICITY CONSUMPTION ---
Enter electricity consumption for Monday (kWh): 0
Enter electricity consumption for Tuesday (kWh): 0
Enter electricity consumption for Wednesday (kWh): 0
Enter electricity consumption for Thursday (kWh): 0
Enter electricity consumption for Friday (kWh): 0
Enter electricity consumption for Saturday (kWh): 0
Enter electricity consumption for Sunday (kWh): 0

--- DAILY CONSUMPTION REPORT ---
Monday : 0.0 kWh
  Status: Normal consumption
Tuesday : 0.0 kWh
  Status: Normal consumption
Wednesday : 0.0 kWh
  Status: Normal consumption
Thursday : 0.0 kWh
  Status: Normal consumption
Friday : 0.0 kWh
  Status: Normal consumption
Saturday : 0.0 kWh
  Status: Normal consumption
Sunday : 0.0 kWh
  Status: Normal consumption

--- WEEKLY SUMMARY ---
Total weekly consumption: 0.0 kWh
Average daily consumption: 0.0 kWh
Highest consumption: 0.0 kWh
Day with highest consumption: Monday
Number of high-consumption days: 0
Weekly analysis completed.
```

10. Conclusion

The Weekly Electricity Consumption Matrix program demonstrates how Python can be used to record daily electricity readings, calculate weekly totals, determine average consumption, and identify high-consumption days.

This case study uses lists, loops, conditional statements, variables, arithmetic operations, and input/output. It demonstrates how basic programming can support electricity usage monitoring and help users recognize days when consumption is unusually high.